CS-768 (Learning with Graphs) Course Project Report

GraphTransLWG: Hybrid GNN–Transformer for Long-Range Graph Reasoning

Eshaan Pareek (22B3986) Tejas Sharma (22B0909)
Aditya Vema Reddy Kesari (22B3985) Matam Kushaal (23B1290)

1 Introduction

Graph Neural Networks (GNNs) aggregate neighborhood information using message passing, but depth increases can cause oversmoothing and oversquashing. This limits long-range dependency modeling. Our project studies a GraphTrans-style hybrid architecture that interleaves local graph convolutions with global self-attention to capture both local topology and distant interactions.

2 Problem Setup

Given a graph $G = (V, E)$ with node features $X \in \mathbb{R}^{|V| \times d_x}$ and graph label y, learn $f_\theta(G) \to \hat{y}$ for graph-level prediction.

Message Passing (local bias). A generic layer updates node i as:

$$\mathbf{h}_i^{(\ell+1)} = \phi\left(\mathbf{h}_i^{(\ell)},\ \square_{j \in \mathcal{N}(i)} \psi(\mathbf{h}_i^{(\ell)}, \mathbf{h}_j^{(\ell)}, \mathbf{e}_{ij})\right),$$

where $\square$ is an aggregation operator.

Global attention (long-range context). With hidden matrix $H \in \mathbb{R}^{n \times d}$:

$$Q = HW_Q, \quad K = HW_K, \quad V = HW_V,$$

$$\mathrm{Attn}(H) = \mathrm{softmax}\left(\frac{QK^\top}{\sqrt{d_h}}\right) V.$$

This enables every node to interact with every other node in one layer.

3 Architecture

The implemented model follows:

1. Input projection: $X \mapsto H^{(0)} \in \mathbb{R}^{n \times d}$.
2. Add a graph-level `[CLS]` token per graph for pooling.
3. Repeat for L blocks:
   - GNN sublayer for local structure encoding.
   - Multi-head self-attention sublayer for global context.
   - Position-wise MLP sublayer.
   - Residual paths and normalization in transformer stack.
4. Readout from `[CLS]` embedding and linear head for logits.

4 Implementation Mapping to Repository

- `models/full_model.py`: `GraphTransConfig`, `GraphTransModel`, CLS pooling path.
- `models/transformer.py`: transformer block composition.
- `models/gnn.py`: local message-passing layers.
- `models/attention.py`: attention mechanism implementation.
- `models/train.py`: training loop with optimizer/loss/backprop.
- `trainers/base_trainer.py`: generic trainer with grad clipping + scheduler hooks.
- `trainers/flag_trainer.py`: FLAG adversarial perturbation training.

5 Training Objective and Optimization

For classification, with logits $z_g = f_\theta(G_g)$ and labels y_g, we minimize:

$$\mathcal{L}(\theta) = \frac{1}{B} \sum_{g=1}^{B} \mathrm{CE}(z_g, y_g).$$

Training uses Adam-family optimization with optional gradient clipping:

$$\theta \leftarrow \theta - \eta \, \nabla_\theta \mathcal{L}.$$

5.1 FLAG Robust Training

FLAG injects adversarial feature perturbations δ in hidden/node input space:

$$\delta^{t+1} = \Pi_\epsilon \left(\delta^t + \alpha \, \mathrm{sign}(\nabla_\delta \mathcal{L}(\theta, \delta^t)) \right).$$

Model parameters are updated against perturbed examples, improving robustness and regularization.

6 Complexity Discussion

Let $n = |V|$, hidden width d, edges $m = |E|$.

- GNN layer cost is typically $\mathcal{O}(md)$.
- Full attention cost is $\mathcal{O}(n^2 d)$ in time and $\mathcal{O}(n^2)$ in memory for attention scores.

Hence, hybrid architectures trade better long-range modeling for higher cost on large graphs.

7 Benchmarks and Experimental Scope

Configuration files indicate evaluation on OGBG-Code2, OGBG-MolPCBA, NCI1, and NCI109 with multiple variants (GCN/GIN/PNA with and without virtual node settings).

8 Conclusion

This project establishes a technical and reproducible foundation for graph representation learning with GraphTrans-style hybrids. The key benefit is combining local inductive bias from message passing with global receptive fields from attention.